Class 19: ML Experiment Tracking, Registry, Evaluation, and Decision-Making

Data-driven Systems Engineering

Roadmap of the class

1. **Record the experiment**
   Runs, configurations, metrics, seeds, data versions, and environments.
2. **Search fairly**
   Baselines, reproducible comparisons, and hyperparameter sweeps.
3. **Manage artifacts**
   The distinct roles of source code, run records, model artifacts, and a registry.
4. **Evaluate for the task**
   Metrics, class imbalance, error inspection, and decision costs.
5. **Make evidence actionable**
   Model interfaces, decision rules, promotion, and rollback.
6. **Connect to practice**
   The wine project, its artifacts, and the hand-off to software engineering.

What is Weights & Biases (W&B)?

A platform for **tracking, comparing, and versioning machine learning work**. A few lines of the `wandb` Python library send metrics, configuration, and files to a web dashboard, which can be hosted by W&B or self-hosted. Website: wandb.ai.

Run	One execution of a script or notebook, with a unique identifier.
Config	The inputs of a run: hyperparameters, seed, dataset version.
Metrics	Values logged with `run.log`: scores, losses, timings, plots.
Sweep	An automated hyperparameter search (grid, random, or Bayesian).
Artifact	A versioned file or directory (dataset, model) with its lineage.
Registry	Curated collections of approved model versions.

In one sentence

Git remembers how the code changed; W&B remembers what each run of that code produced.

Alternatives: MLflow (open source, self-hosted), TensorBoard (local dashboards), Comet, ClearML.

W&B case study: search space and protocol

Search space of a gradient boosting regressor:

Hyperparameter	Values
`n_estimators`	100, 200, 300
`max_depth`	3, 5, 7
`learning_rate`	0.05, 0.1, 0.2
`min_samples_split`	2, 5, 10

Protocol:

- Data: red wine quality, 1,599 samples; target `quality`.
- Split: 75% training (1,199 wines), 25% test (400 wines), fixed seed.
- Grid search: $3^4 = 81$ configurations (one W&B run each) $\times$ 5-fold cross-validation $=$ 405 fits, scored by R^2 on the training split; each run then refits once to save its artifact.
- The test split is used once, after selection.

Implemented in the `wine-wandb` project (`configs/sweep.yaml`, `src/wine_wandb/sweep.py`); no numerical result of the sweep is shown until it has been executed.

W&B: the record behind an experiment

The experiment tracker links an explicit hypothesis to a reproducible configuration and measurable evidence. An example record for the wine-quality exercise:

Field	Example logged information
`run.config`	model family, grid, random seed, cross-validation scheme
`run.summary`	mean CV R^2, CV standard deviation, fitting time
`git commit`	commit SHA for training and preprocessing code
`dataset`	dataset identifier, checksum or immutable artifact version
`model artifact`	serialized `Pipeline` (preprocessing + predictor)

- Log candidate performance using cross-validation on the training partition; evaluate the chosen model only once on the untouched test partition.
- **Do not confuse** experiment logs, artifact versions, and model approval: they answer different questions.

W&B: minimal experiment tracking in Python

```python
import wandb
from sklearn.model_selection import cross_val_score

config = {"model": "GradientBoostingRegressor", "seed": 42}
with wandb.init(project="wine-quality", config=config) as run:
    scores = cross_val_score(pipeline, X_train, y_train, cv=5)
    run.log({"cv/r2_mean": scores.mean(),
             "cv/r2_std": scores.std()})
    artifact = wandb.Artifact("wine-pipeline", type="model")
    artifact.add_file("pipeline.joblib")
    run.log_artifact(artifact)
```

Artifacts, versioning, and registry

How does a trained model become an approved, versioned, reversible release?

Git versus artifacts, the W&B Registry, promotion and rollback

1. Modeling | 2. Tracking | **3. Registry** | 4. Evaluation | 5. Decisions | 6. Practice

Git, tracking, artifacts, and registry: distinct roles

Component	Stores or identifies	Primary question
GitHub	source, tests, configuration, CI workflows	Which code produced this run?
W&B Runs	parameters, metrics, environment and run identifiers	What happened during training?
W&B Artifacts	immutable versions of data and model files	Which exact file was produced or used?
W&B Registry	curated collections and approved model versions	Which version is eligible for operational use?

Key distinction

Versioning a `model.pkl` filename is not enough: a deployed model needs *code* + *data* + *preprocessing* + *environment* + *trained artifact* + *approval record*.

Why not commit trained binaries directly to Git?

Git is excellent for

- code and configuration diffs;
- review and pull requests;
- reproducible workflow definitions.

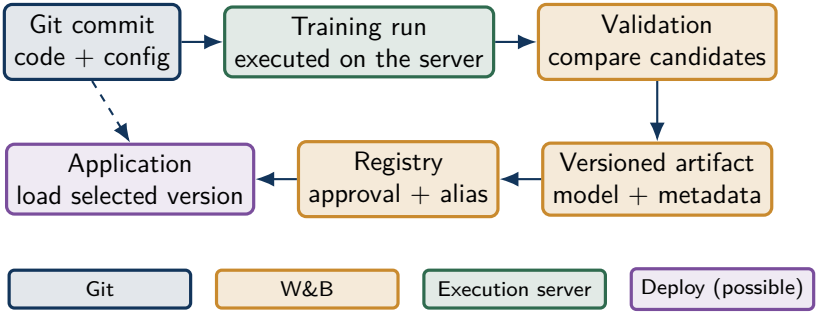
Model artifacts additionally need

- versioned object storage and integrity hashes;
- links to runs and input datasets;
- approval, access control, and rollback history.

Practical exception

Tiny demonstration binaries may be committed for a self-contained example, but this is not a scalable production artifact strategy. Git LFS manages large files differently; it does not by itself provide experiment tracking or model governance.

From an experiment to an approved model



W&B: publish an artifact to the Registry

```python
import wandb
import joblib

joblib.dump(pipeline, "pipeline.joblib")
with wandb.init(project="wine-quality") as run:
    artifact = wandb.Artifact("wine-pipeline", type="model")
    artifact.add_file("pipeline.joblib")
    logged = run.log_artifact(artifact)
    logged.wait()
    run.link_artifact(
        logged,
        target_path="wandb-registry-model/wine-quality",
    )
```

Promotion, rollback, and a model release contract

1. **Candidate:** compare runs on validation data; preserve the full candidate record.
2. **Gate:** test metrics, stability, inference latency, feature schema, and reproducibility against agreed criteria.
3. **Approval:** link the accepted artifact to a Registry collection; assign a documented alias such as `champion` when appropriate.
4. **Release:** application records the resolved immutable artifact version and environment.
5. **Rollback:** repoint the release to the preceding approved artifact and revalidate compatibility.

Discussion

Why is selecting the highest CV score insufficient to promote a model to production?

Evaluation

Which metric tells the truth about this task?

Metrics, class imbalance, inspecting the errors

1. Modeling | 2. Tracking | 3. Registry | **4. Evaluation** | 5. Decisions | 6. Practice

Evaluation metrics must match the task

Tracking identifies which run performed best; it does not establish whether that run is adequate for the decision it supports.

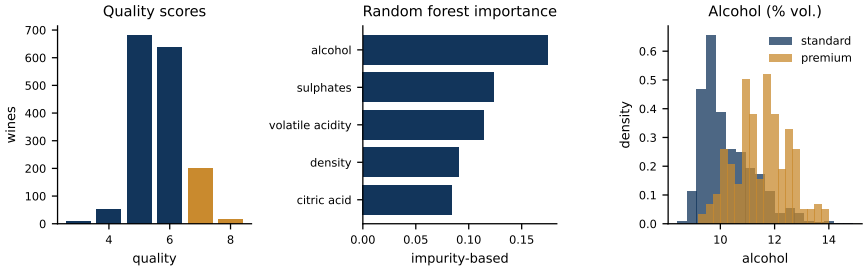
Classification (premium wine)

- **Precision** $= \frac{TP}{TP+FP}$ and **recall** $= \frac{TP}{TP+FN}$, from the counts of true and false positives and negatives.
- **PR-AUC:** area under the precision–recall curve; summarizes all possible cut-offs.
- **Accuracy** alone is misleading when classes are imbalanced.
- **Calibration:** needed when an action depends on the predicted probability.

Regression (wine quality)

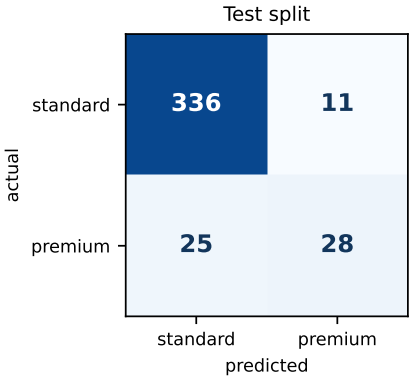
- **MAE** $= \frac{1}{n}\sum_i |y_i - \hat{y}_i|$ and **RMSE** $= \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$, in the unit of the target (quality points).
- **RMSE** penalizes large errors more than MAE does.
- R^2**:** share of the variance explained; 0 for a model that always predicts the mean.
- **Residuals** show where the model fails, for example at extreme scores.

The wine dataset at a glance



- **Imbalance:** 217 of 1,599 wines (13.6%) have quality ≥ 7 and are labelled *premium*; the same split as the regression exercise gives 1,199 training and 400 test wines (164 and 53 premium).
- **Importance:** alcohol, sulphates, and volatile acidity account for 41% of the impurity-based importance of the random forest; importance measures association, not causal effect.
- **Separation:** premium wines have a higher median alcohol content (11.6 versus 10.0% vol.), but the distributions overlap, so no single feature decides the label.

Inspecting the errors



Test split (400 wines), random forest, predicted classes.

$$\text{precision} = \frac{28}{28+11} = 0.72, \qquad \text{recall} = \frac{28}{28+25} = 0.53$$

$$\text{accuracy} = \frac{336+28}{400} = 0.91$$

- 25 of the 53 premium wines are missed (FN), while 11 standard wines are labelled premium (FP).
- The accuracy of 91% sounds good, but always answering "standard" already scores 86.75%; the confusion matrix exposes the missed premium wines.
- Whether 25 missed wines are acceptable depends on the cost of a missed premium wine relative to a wrong label: a business input, not a property of the model.

Accuracy fails under imbalance: alternatives

Wine example

Always answering "standard" scores 86.75% accuracy with a recall of 0. The random forest reaches 91%, only 4 points more, yet misses almost half of the premium wines.

Metric	What it measures	Always "standard"	Random forest
Accuracy	Share of correct answers; dominated by the majority class.	0.87	0.91
F1-score	Harmonic mean of precision and recall, $2PR/(P+R)$; ignores true negatives.	0	0.61
ROC-AUC	Chance that a random premium wine outscores a random standard one; independent of any cut-off.	0.50	0.94
PR-AUC	Area under the precision–recall curve; chance level is the share of positives.	0.13	0.75

- F1 is computed from the predicted classes and weighs both errors equally.
- ROC-AUC (0.94) exceeds PR-AUC (0.75) because true negatives dominate; with rare positives, prefer PR-AUC.
- None encodes the cost of each error: that is a business input.

From prediction to decision

How does a score become an action that people can trust?

Interfaces, the decision rule, decision support, where trust breaks

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | **5. Decisions** | 6. Practice

A registered model creates value through an interface

- A registered trained model that cannot be invoked or interpreted has limited practical value, regardless of its offline metrics.
- An interface makes predictions available to users, services, and operational workflows.
- The interface also determines trust: input validation, response format, and clarity matter as much as the metric used to select the model.

A prediction is not a decision

- A premium score suggests that a wine deserves a tasting, not a final grade; a process alert suggests investigation, not automatic shutdown.
- A classifier estimates $P(y = 1 \mid \mathbf{x})$; the class it outputs applies a decision rule, $\hat{y} = \mathbf{1}[\hat{p}(\mathbf{x}) \geq 0.5]$.
- `predict` makes that rule implicit: it assumes both errors cost the same.

In this class

We adopt the predicted class as the decision. With unequal error costs, the cut-off would be a separate design choice.

Example: a decision-support interface for wine



- The interface loads the registered `wine-premium` artifact, so preprocessing cannot diverge from training, and shows the predicted label together with the estimated probability.
- The predicted class is the decision; the probability tells the user how confident the model is.
- The interface should identify the exact artifact version it loaded.

A decision-support interface: requirements and failure modes

What it must provide

- What the model does and does not do.
- Inputs that are meaningful domain variables, not technical internals.
- Outputs that support action: label, probability, explanation.
- Provenance: data version, model artifact version, monitoring signals.

Where trust breaks down

- Uncertainty hidden behind one precise-looking number.
- Inputs that violate the assumptions of training.
- A decision rule adopted by default and never revisited.
- A demonstration model that looks authoritative without validation.

Practice and wrap-up

How do the pieces fit together in a real repository?

Repository connections and take-away

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | 5. Decisions | 6. Practice

Repository connections

Repo activity

- The `wine-wandb` project (a separate repository): the complete wine-quality project used throughout this class, with two tasks on the same dataset.
- `configs/`: `baseline.yaml` and `sweep.yaml` (regression), `classifier.yaml` (premium wine).
- `train.py`, `sweep.py` and `train_classifier.py`: tracked runs; each logs the dataset checksum, the Git commit, and a pipeline artifact (`wine-pipeline`, `wine-premium`).
- `evaluate.py`, `register.py`, `figures.py`: one-time test evaluation, registry promotion, and the figures of this class.

Papers and materials: experiments and provenance

Papers and materials

- Bergstra, J., Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Zaharia et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45.
- Sculley et al. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28.
- scikit-learn User Guide: hyperparameter tuning and model selection.
- Tool documentation: Weights & Biases and MLflow Tracking.

Papers and materials: evaluation and decisions

Papers and materials

- Saito, T., Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot for imbalanced binary classifiers. *PLoS ONE*, 10(3), e0118432.
- Niculescu-Mizil, A., Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of ICML*, 625–632.
- Mitchell et al. (2019). Model cards for model reporting. *Proceedings of FAT**, 220–229.
- scikit-learn User Guide: model evaluation, including metrics, calibration, and threshold-based decisions.
- Tool documentation: W&B Model Registry and MLflow Model Registry.

Take-away message

A model is selected by reproducible experiments,
preserved as a versioned artifact, and approved in a Registry.

Its operational decision also depends on
evaluation evidence, the release configuration, and error costs.

Thank you for your attention.

Next: Class 20 — introduction to software engineering for data products.